Akademia Górniczo- Hutnicza
Wydział Informatyki



Ćwiczenie laboratoryjne

Prosty mikroprocesor i system komputerowy realizowany w logice programowalnej

Nazwa kodowa: **cpu**. Wersja **20261020**

dr inż. Ada Brzoza
ada.brzoza@agh.edu.pl
Adaptacja do Vivado 2024.1 – dr inż. Wojciech Zaborowski

1. Wstęp

1.1 . Cel ćwiczenia

- Poznanie ogólnej zasady funkcjonowania mikroprocesorów,
- Poznanie przykładowego sposobu implementacji mikroprocesora o minimalnej funkcjonalności,
- Zapoznanie się z przykładem implementacji prostego systemu komputerowego (*Simple Computer System* - SCS)
- Sprawdzenie działania zaimplementowanego mikroprocesora i systemu komputerowego na rzeczywistej platformie sprzętowej.

1.2 . Wymagane wiadomości i umiejętności

- Znajomość idei działania mikroprocesora,
- Umiejętność obsługi płytki testowej Digilent ZYBO, dokumentacja: https://reference.digilentinc.com/reference/programmable-logic/zybo/reference-manual
- Umiejętność posługiwania się programem terminalowym.

1.3 . Dodatkowe umiejętności

Wykonanie ćwiczenia pozwoli Studentom na zdobycie także dodatkowych umiejętności:

- Poznanie elementów języka Verilog,
- Podstawy tworzenia konfiguracji dla układów FPGA w środowisku Xilinx Vivado: zestawienie projektu, kompilacja, generowanie plików wynikowych, tworzenie i dostosowywanie modułu przy pomocy narzędzia IP Integrator.

1.4 . Wykorzystywane narzędzia

Ćwiczenie wykonywane jest na płycie testowej ZYBO produkcji Digilent. Płyta jest wyposażona w zintegrowany programator z interfejsem USB. Do tworzenia konfiguracji FPGA używane jest środowisko **AMD/Xilinx Vivado w wersji 2024.1**. Do wykonania ćwiczenia można posłużyć się wersją darmową Vivado ML standard tego środowiska (nie jest potrzebna wersja komercyjna).

Dodatkowo będzie wykorzystany program emulatora terminala (PuTTY, TeraTERM) .

1.5 . Dostępne materiały

Do wykonania ćwiczenia niezbędne są następujące materiały:

- dokumentacja techniczna rdzenia mikroprocesora z pliku ***scs_trm.pdf***,
- projekt startowy dla środowiska Vivado w wersji **2024.1**.

2 . System testowy

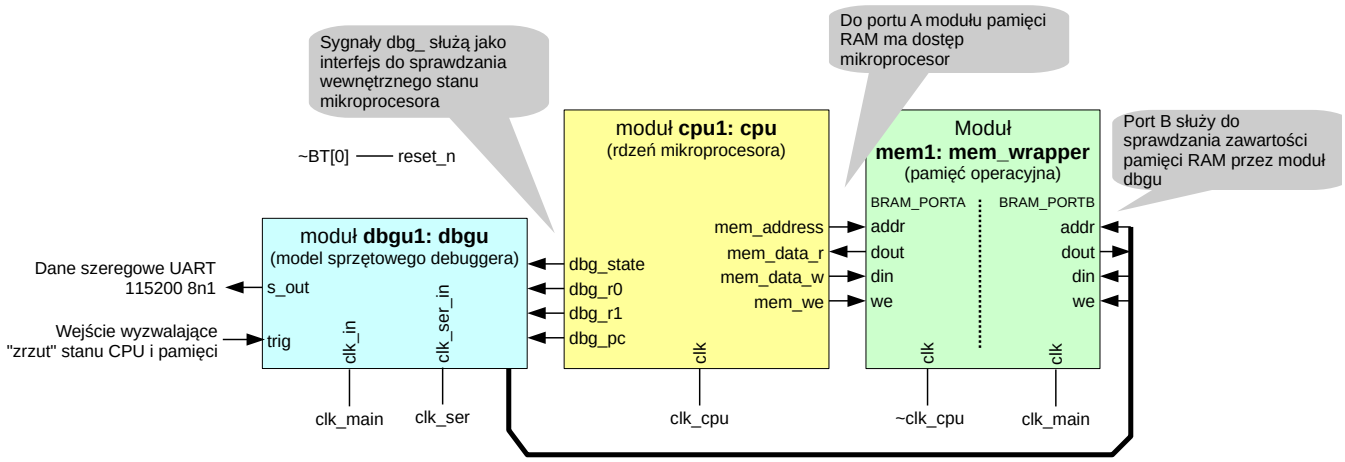
Modułem nadrzędnym systemu testowego jest ***toplevel.v***. Po jego otwarciu warto pobieżnie zapoznać się z jego zawartością. Pewnym ułatwieniem może być opis w dalszej części niniejszego punktu.

2.1 . Ogólny schemat blokowy

W dostępnym projekcie startowym znajduje się rdzeń mikroprocesora oraz podstawowe układy umożliwiające jego działanie:

- blok pamięci operacyjnej,
- układy kondycjonujące sygnały wejściowe z przycisków i przełączników,
- generatory sygnałów zegarowych.

Uproszczony schemat blokowy systemu testowego z projektu bazowego został przedstawiony na poniższym rysunku.



Podstawowymi blokami systemu są mikroprocesor (moduł ***cpu1*** typu ***cpu***, plik ***cpu.v***) oraz jego pamięć operacyjna (moduł ***mem1***, ***mem_wrapper***).

Mikroprocesor posiada trzy typy interfejsów:

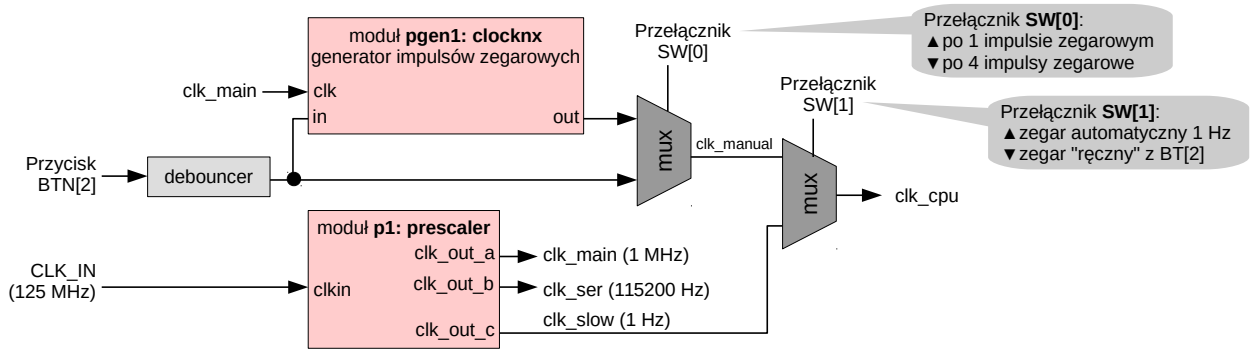
- do sterowania pracą: sygnał ***reset_n*** aktywowany stanem logicznym niskim,
- dostępu do pamięci:
  - ***mem_address*** - do adresowania aktywnej komórki pamięci RAM,
  - ***mem_data_r*** - do odczytu danych z pamięci,
  - ***mem_data_w*** - do zapisu danych do pamięci,
  - ***mem_we*** - wyjście zezwolenia na zapis do pamięci.
- do debugowania:
  - ***dbg_state*** - informujący o aktywnym etapie wykonywania pojedynczej instrukcji (każda instrukcja wykonywana jest w 4 cyklach sygnału zegarowego clk),
  - ***dbg_r0, dbg_r1, dbg_pc*** - informujące o aktualnej zawartości rejestrów R0, R1 i PC (licznika programu).

Moduł pamięci operacyjnej zbudowany jest w oparciu o blok pamięci RAM o dostępie dwuportowym. Oznacza to, że do jednego obszaru pamięci mamy praktycznie niezależny dostęp z dwóch interfejsów oznaczonych na rysunku jako porty BRAM_PORTA i BRAM_PORTB (BRAM od *Block Random Access Memory*). Port A w przykładowym systemie zarezerwowany jest

dla mikroprocesora, natomiast port B służy do obserwowania zawartości wybranej komórki pamięci RAM.

2.2 . Zarządzanie sygnałami zegarowymi

Schemat podsystemu dystrybucji sygnałów zegarowych w projekcie testowym został przedstawiony i pokrótce opisany na poniższym rysunku.



Do wyboru aktywnego źródła sygnału zegarowego służą przełączniki SW[0] i SW[1].

- Jeśli przełącznik SW[1] ustawiony jest w pozycji „w dół”, wówczas sygnał zegarowy podajemy ręcznie, wciskając przycisk BTN[2].
- Ustawienie przełącznika SW[1] na pozycję „do góry” spowoduje automatyczne generowanie powolnego sygnału zegarowego o częstotliwości 1 Hz.
- Jeśli ustawimy przełącznik SW[0] w pozycji „w dół”, wówczas ręczny sygnał zegarowy będzie podawany po 4 impulsy za każdym wciśnięciem BTN[2], co ułatwia obserwację pracy mikroprocesora wykonującego pełne **cykle instrukcji**.
- Jeśli ustawimy przełącznik SW[0] w pozycji „w górę”, wówczas ręczny sygnał zegarowy będzie podawany po jednym impulsie za każdym wciśnięciem BTN[2]. Wtedy można obserwować, co dzieje się w mikroprocesorze podczas wykonywania poszczególnych **cykli zegarowych**.

2.3 . Program testowy

W pamięci RAM systemu znajduje się prosty program testowy. Został on przedstawiony w poniższej tabeli.

Adres	Instrukcja	Kod bin.	Kod hex.
00	mov r0, #0	1000 0000	80
01	mov r1, #0	1001 0000	90
02	mov r0, #1	1000 0001	81
03	mov r1, #2	1001 0010	92
04	mov r0, r1	1010 0000	A0
05	jmp #0	0000 0000	00

Kody operacji (kolumny „Kod bin.” i „Kod hex.”) znajdują się w kolejnych komórkach pamięci RAM o adresach wyszczególnionych w lewej kolumnie. Wyjaśnienie mnemoników i kodowania poszczególnych instrukcji można znaleźć w dokumentacji mikroprocesora (***cpu_dok.pdf***).

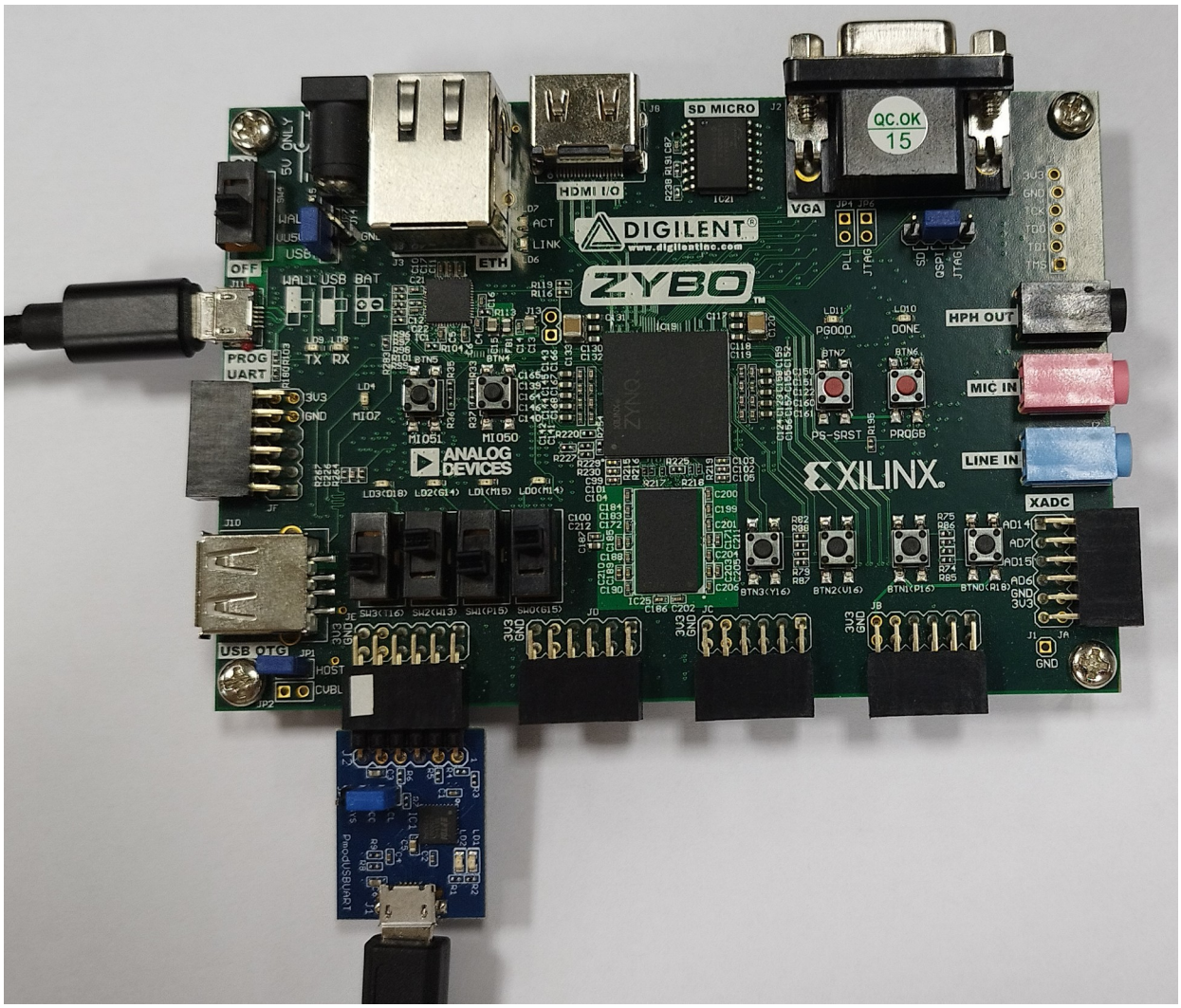
Program testowy do wykonania musi zostać umieszczony w pliku o rozszerzeniu ***.coe*** (z odpowiednią składnią) i wskazany w edytorze schematów blokowych środowiska Vivado. Bardziej szczegółowy opis procedury dodawania i uaktualniania kodu programu znajduje się w dalszej części instrukcji.

3 . Wykonanie ćwiczenia

3.1 . Sprawdzenie podłączenia sprzętu

Płytka testowa Zybo powinna być podłączona do komputera PC dwoma przewodami:

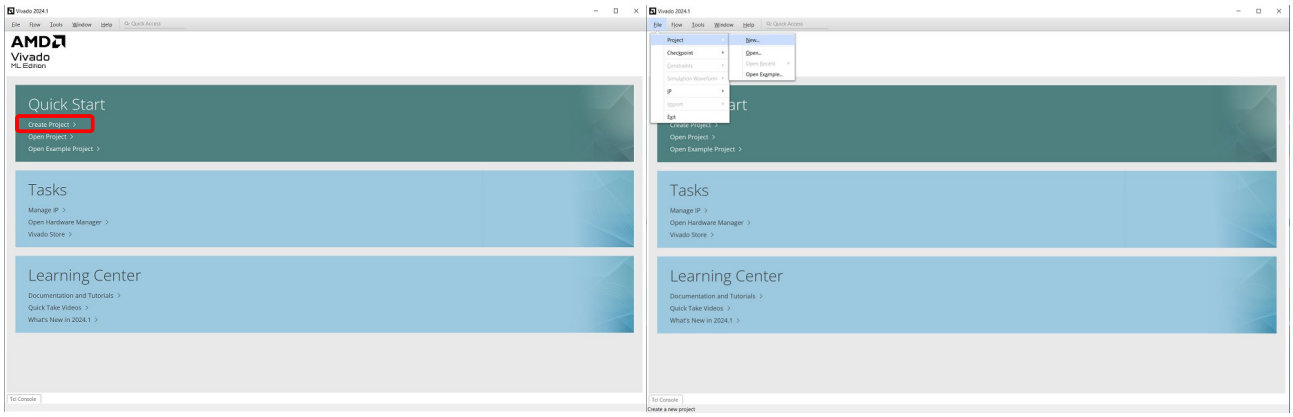
1. Przewodem USB-micro ze złącza PROG/UART do gniazda USB w komputerze PC,
2. Przewodem USB-micro pomiędzy podłączonym do gniazda JE modułem PMOD USB UART do drugiego gniazda USB komputera PC.



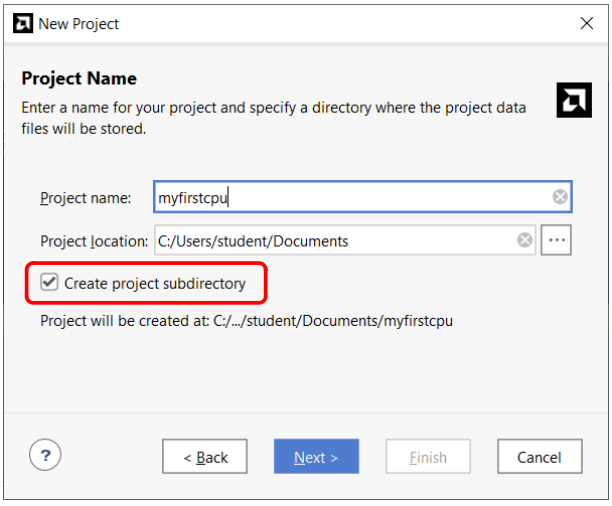
3.2 . Utworzenie systemu mikroprocesorowego w FPGA

Zaczynamy od pobrania i wypakowania kodów źródłowych modułów opisanych w języku opisu sprzętu Verilog. Zostały one dołączone do materiałów umieszczonych w kursie na platformie UPEL. Następnie otwieramy środowisko **Vivado 2024.1**.

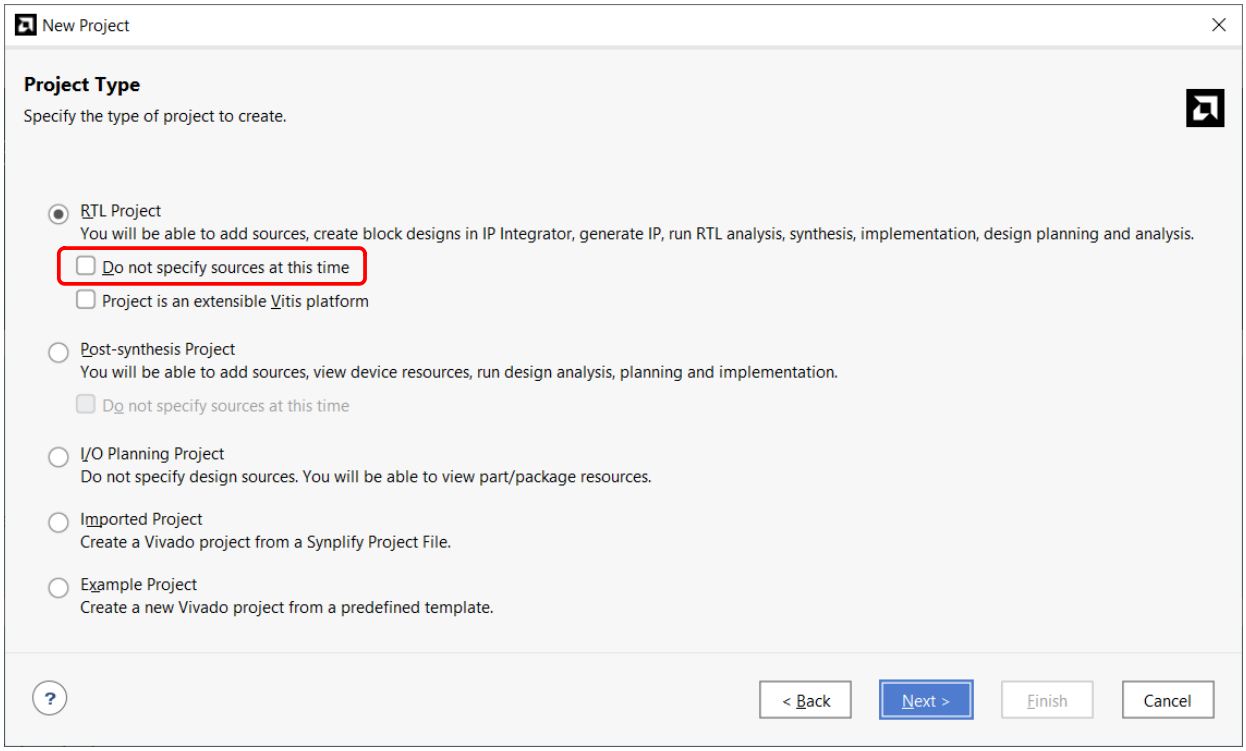
1. Wprost z ekranu powitalnego wybieramy polecenie ***Create project*** > lub z menu głównego Vivado wybieramy ***File → Project→New...***



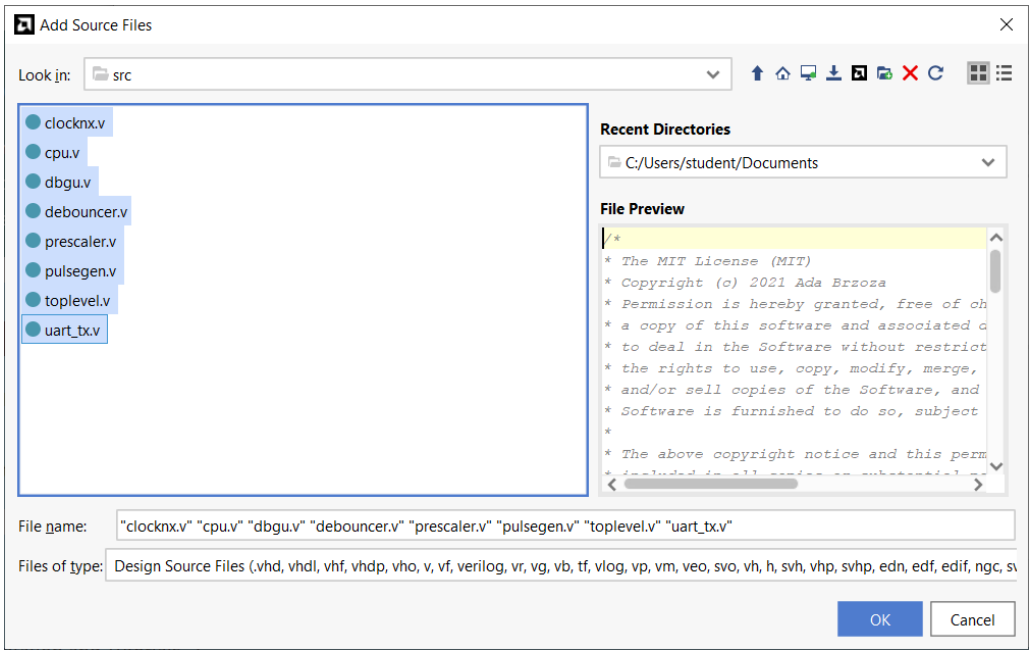
2. Uruchomi się kreator tworzenia nowego projektu. W pierwszym ekranie przechodzimy od razu dalej przyciskiem ***Next***.
3. W drugim oknie kreatora wybieramy ścieżkę i nazwę projektu. Warto zaznaczyć pole ***Create project subdirectory***.



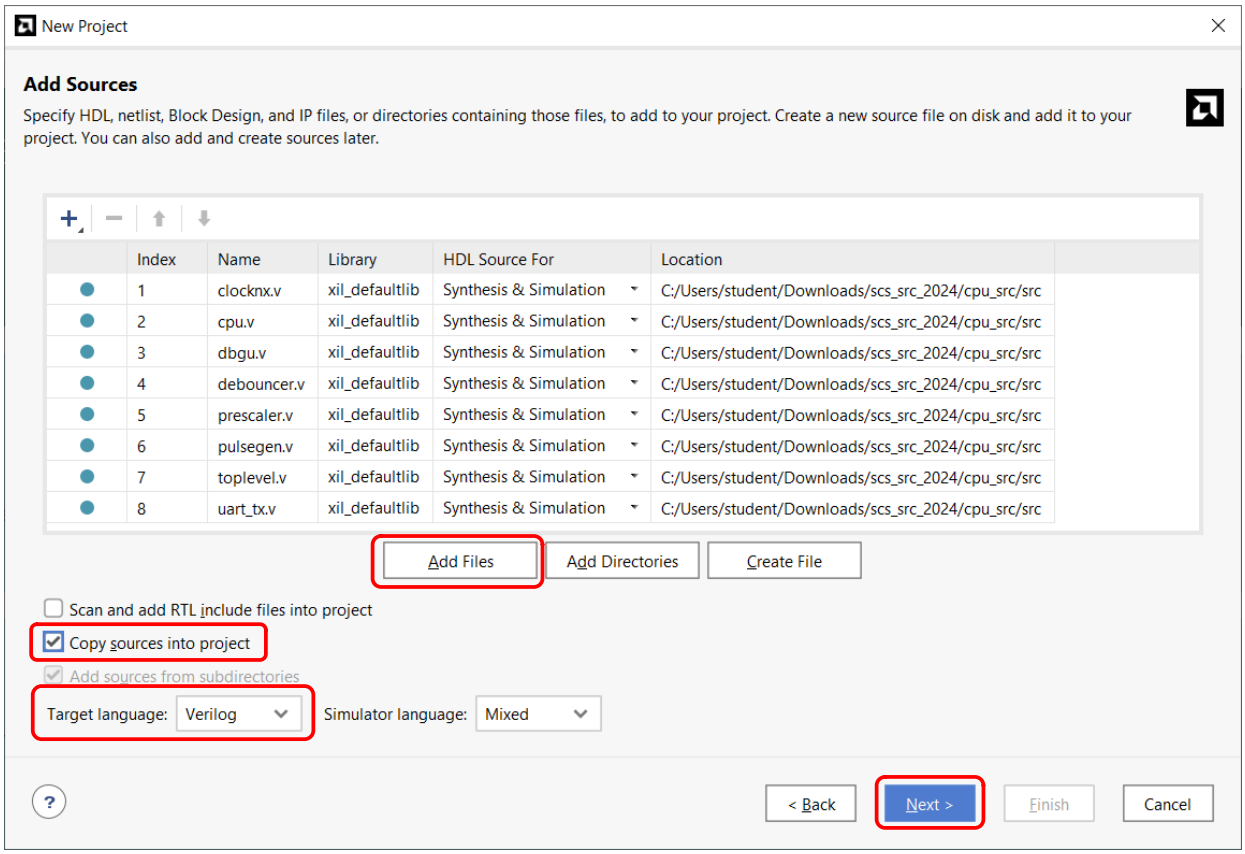
4. W kolejnym oknie wybieramy typ projektu: tutaj wybierzemy podstawowy typ, czyli ***RTL Project***. Dodatkowo pilnujemy, aby **nie była** zaznaczona opcja ***Do not specify sources at this time*** – pozwoli to nam dodać gotowe pliki źródłowe na etapie tworzenia projektu. Klikamy ***Next***.



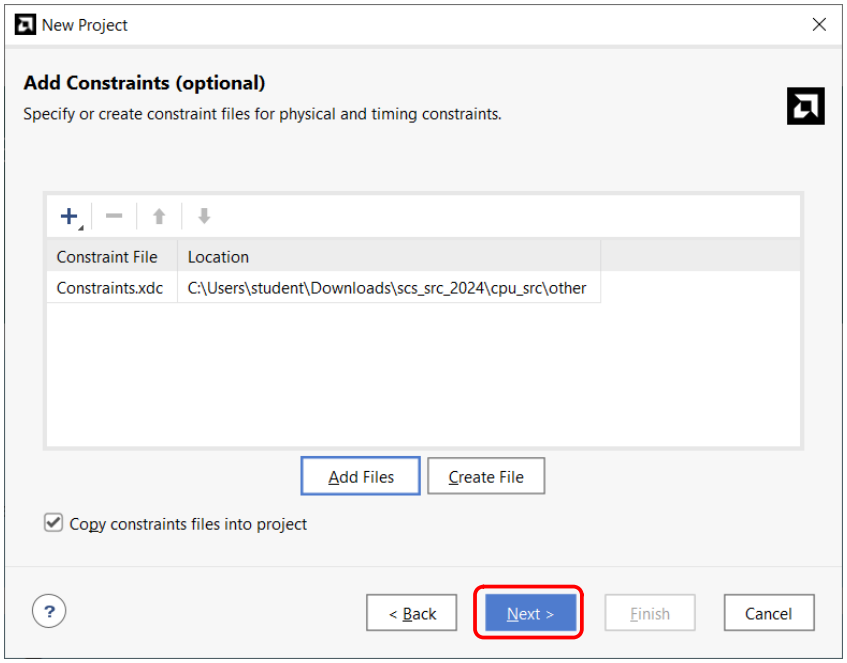
5. W kolejnej części kreatora możemy dodać gotowe pliki źródłowe. Klikamy przycisk ***Add Files*** i wskazujemy wszystkie pliki z podkatalogu ***src*** . Zatwierdzamy ***OK***.



6. Sprawdzamy, aby zaznaczona była opcja ***Copy sources into project***. Oprócz tego wybieramy domyślny język (***Target language***): ***Verilog***, ***Simulator language*** pozostawiamy na ***Mixed***. Przechodzimy dalej przyciskiem ***Next***.



7. Ekran kreatora *Add Existing IP* na razie pomijamy przechodząc dalej przyciskiem ***Next***.
8. Następnie pojawi się ekran kreatora *Add Constraints (optional)*. Również wciskamy przycisk ***Add Files*** i wskazujemy plik ***Constraints.xdc*** z podkatalogu ***other***. Przechodzimy dalej przyciskiem ***Next***.

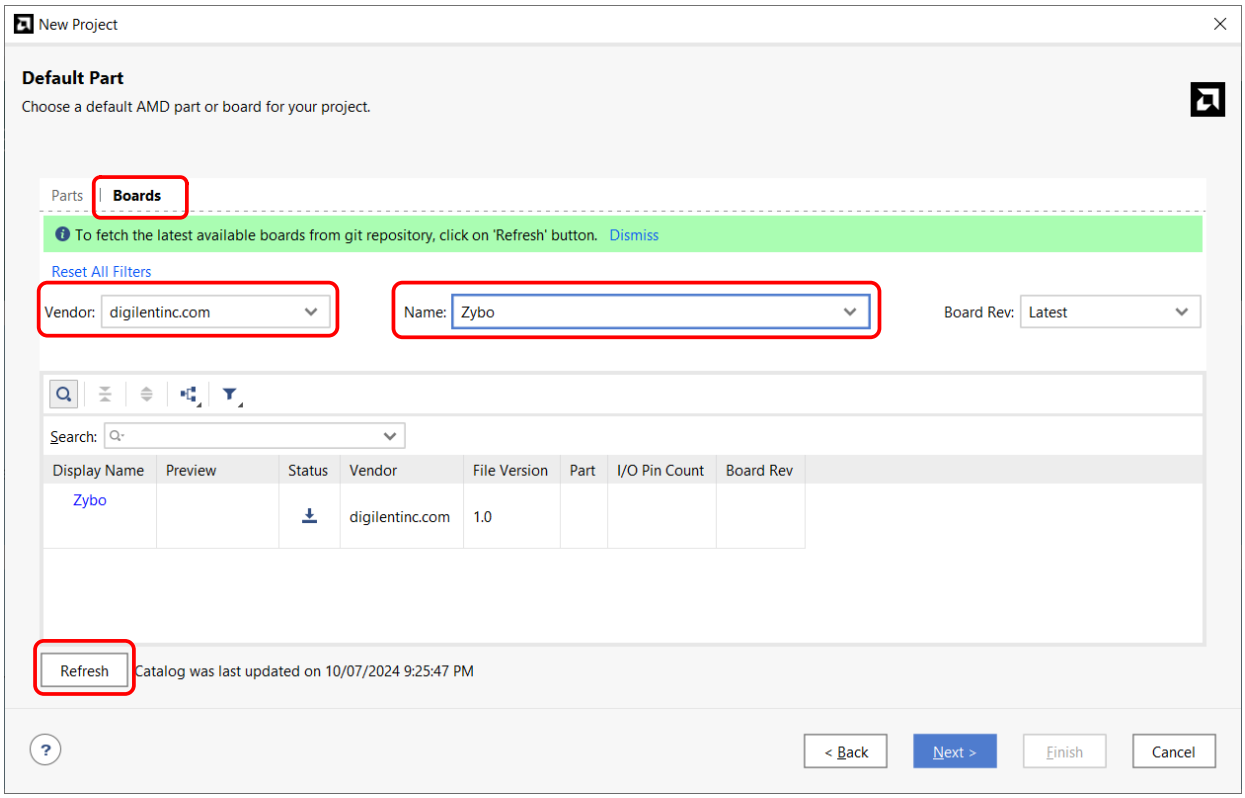


9. W ekranie *Default Part* klikamy na zakładkę ***Boards***, a następnie wybieramy Vendor***:*** ***digilentinc.com*** i Name: ***Zybo***.

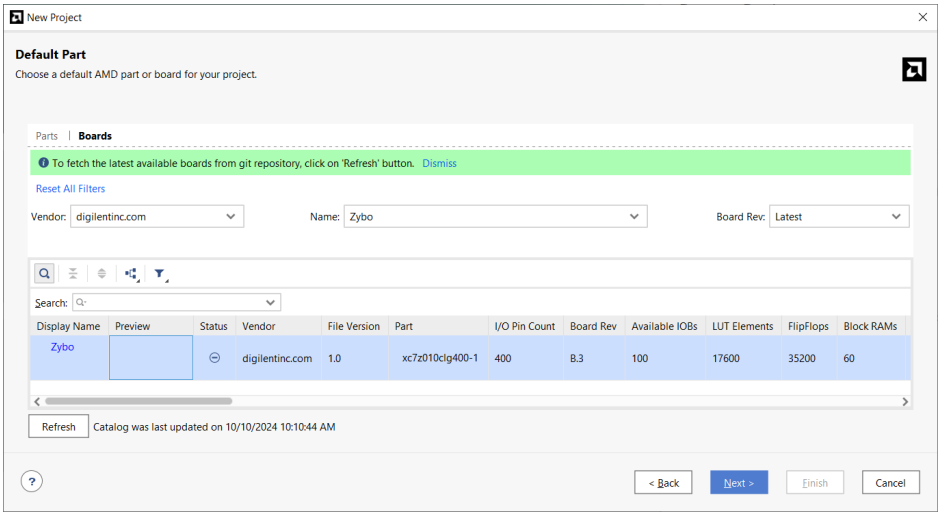
   a) **Uwaga 1: samo pojawienie się płytki Zybo na liście oznacza wyłącznie „odfiltrowanie" jej nazwy. Aby została ona rzeczywiście wybrana, należy kliknąć na wiersz w tabeli, w której została wyświetlona – w ten sposób należy ją podświetlić i wtedy dopiero można iść dalej.** Jeśli nie dopilnujemy tego kroku teraz, to środowisko Vivado wybierze płytkę domyślną i nie poinformuje nas o tym ani teraz, ani w czasie realizacji kolejnych kilku kroków, tylko już przy ostatecznym generowaniu pliku wynikowego bitstream. Dlatego, w razie wątpliwości, można zwrócić się do osoby prowadzącej – wtedy możemy uniknąć późniejszej żmudnej edycji projektu lub nawet zestawiania projektu od nowa.

   b) **Uwaga 2:** Jeśli opcje te nie są dostępne, to należy przyciskiem **Refresh** odświeżyć listę wspieranych płytek (nie należy robić tego bez potrzeby, gdyż proces aktualizacji listy trwa dłuższa chwilę.) W przypadku widoku jak na rysunku poniżej warto sprawdzić w kolumnie status czy pakiet wsparcia dla wybranej płytki jest zainstalowany. Jeśli nie, to należy go doinstalować.)

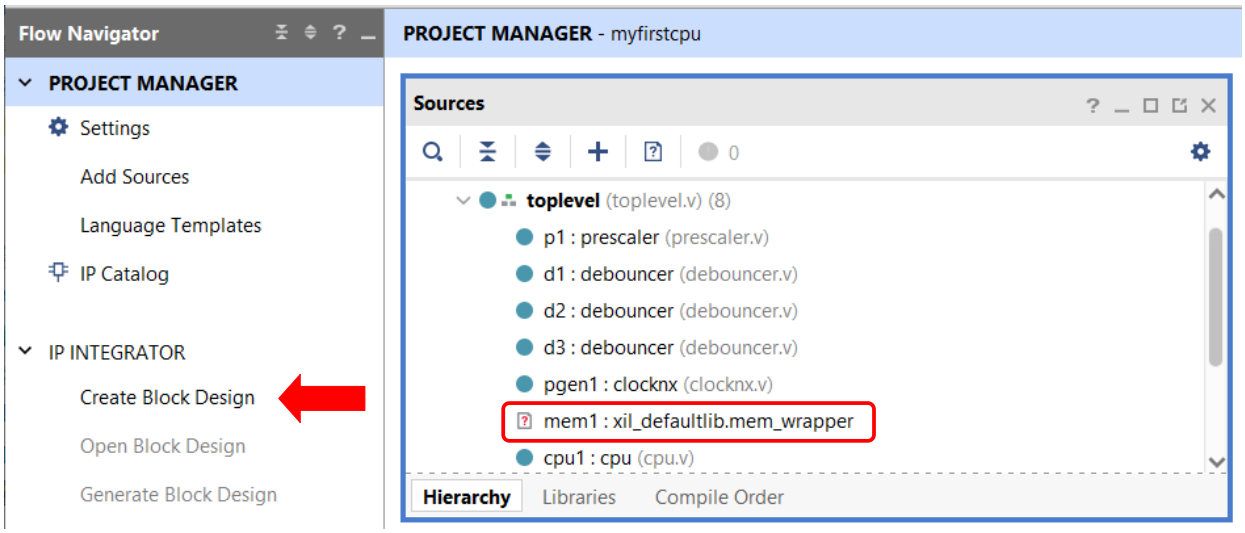
   Dzięki temu etapowi środowisko Vivado zostanie poinformowane m.in. o typie układu FPGA, dla którego będziemy tworzyć konfigurację.



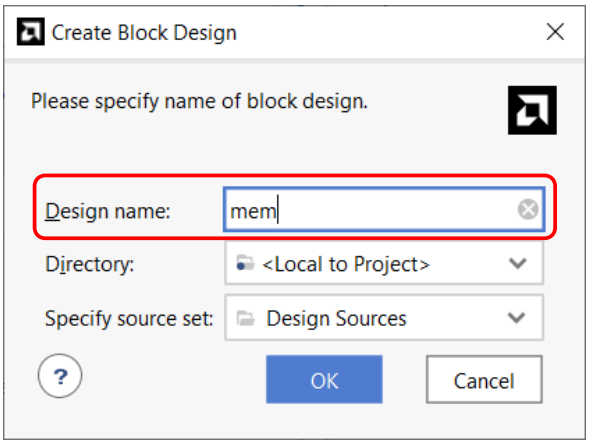
Przed pójściem dalej należy zwrócić uwagę, czy faktycznie wsparcie dla płytki jest zainstalowane i płytka została wybrana (pozycja *Zybo* powinna być podświetlona jak na rysunku poniżej/dalej) ↓↓↓↓↓↓↓↓



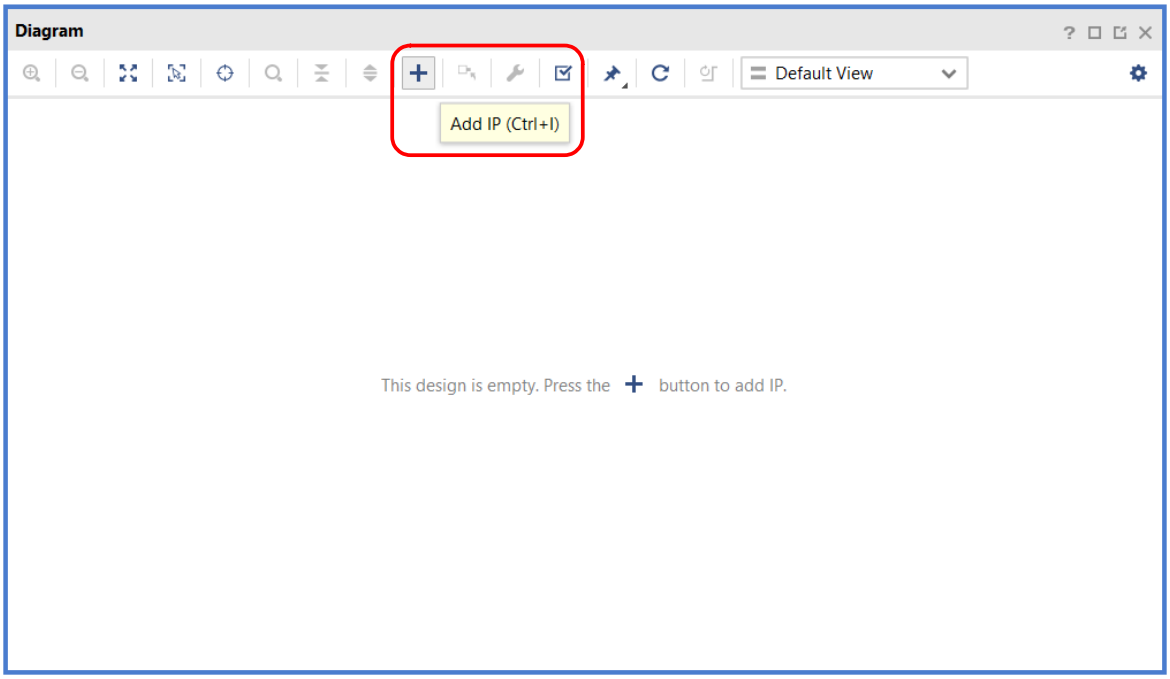
10. Ostatni krok kreatora nowego projektu to podsumowanie. Zatwierdzamy je przyciskiem ***Finish***.
11. Pojawi się główne okno środowiska Vivado. W części *Project Manager* w zakładce *Hierarchy* możemy dostrzec, że przy elemencie *mem1* typu *xli_defaultlib.mem_wrapper* pojawił się znak zapytania – tego modułu brakuje w projekcie i jest to pamięć, którą wygenerujemy przy pomocy podsystemu *IP Integrator* środowiska Vivado.



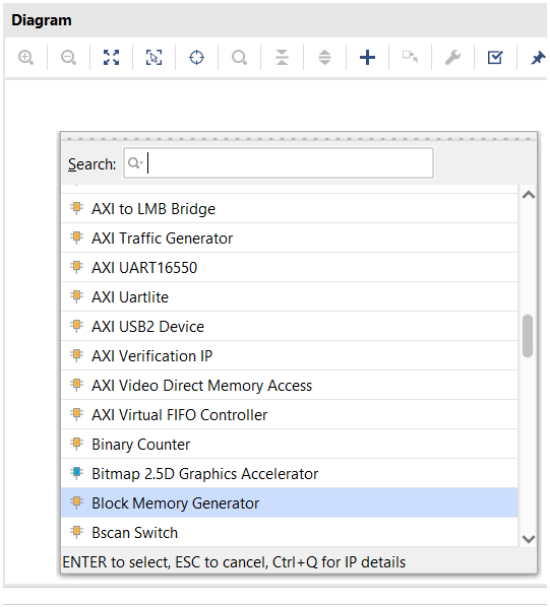
12. W polu ***Flow Navigator*** po lewej stronie okna głównego pod *IP Integrator* klikamy ***Create Block Design***. W polu *Design name* wpisujemy ***mem*** i zatwierdzamy przyciskiem *OK*.



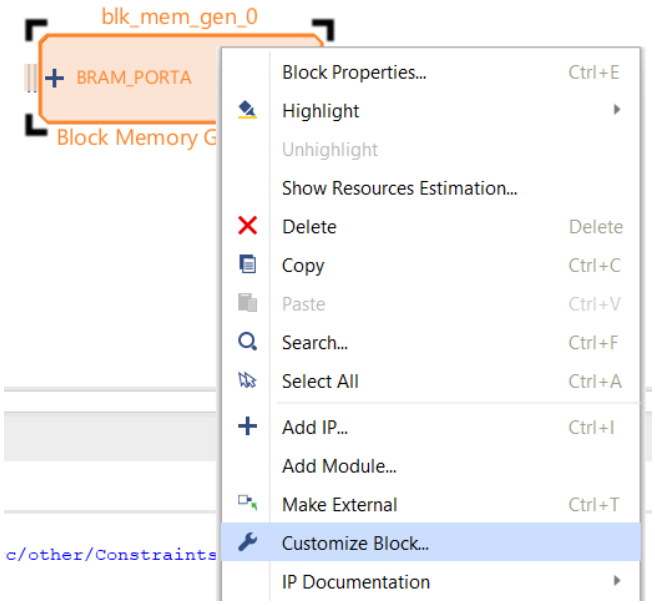
13. Otworzy się podsystem tworzenia schematu blokowego. Na razie obszar rysowania jest pusty. W oknie edycji schematu *Diagram* wskazujemy ikonkę ***Add IP***.



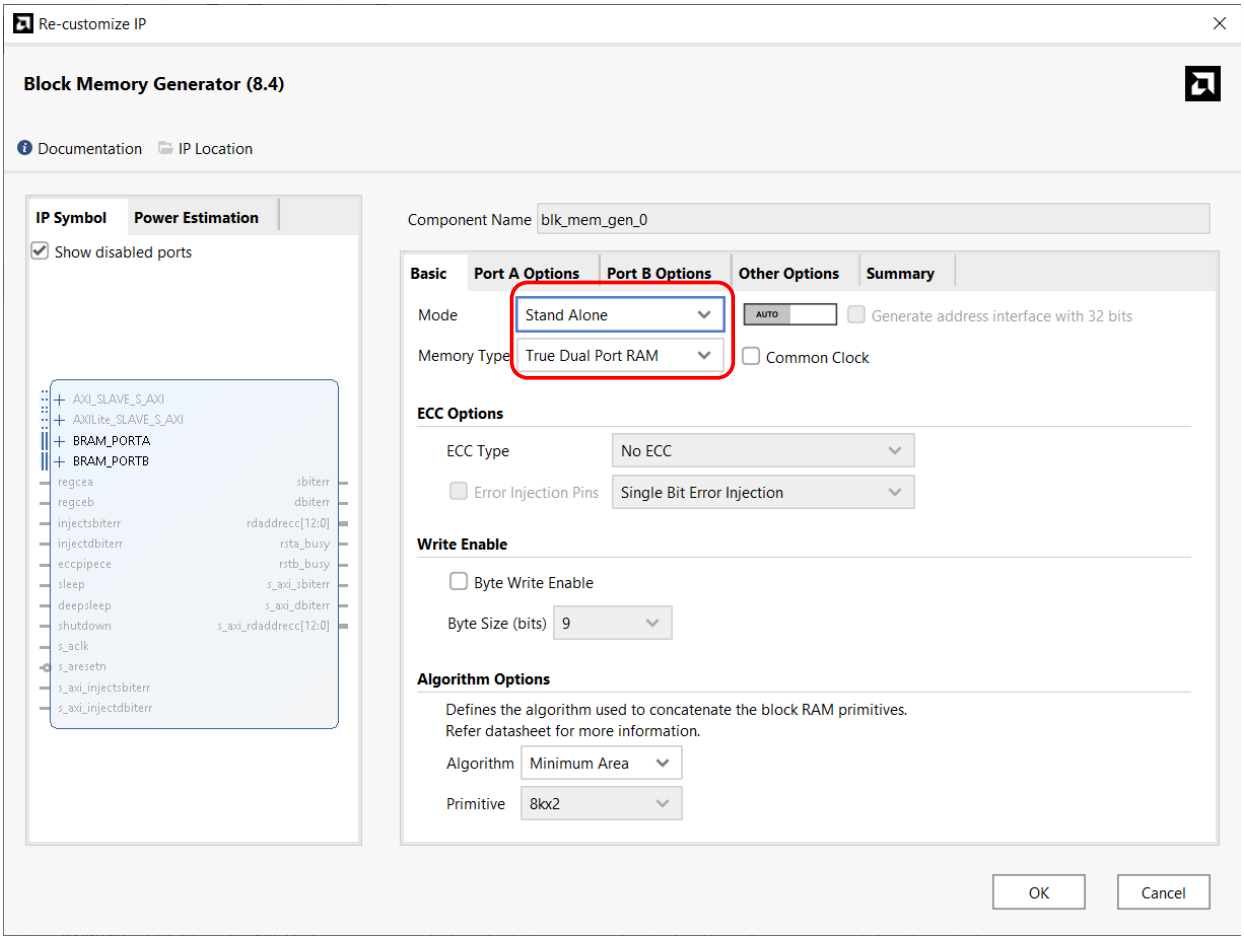
14. Na liście dostępnych modułów odnajdujemy element ***Block Memory Generator***. Klikamy go dwa razy i czekamy na dodanie do schematu blokowego.



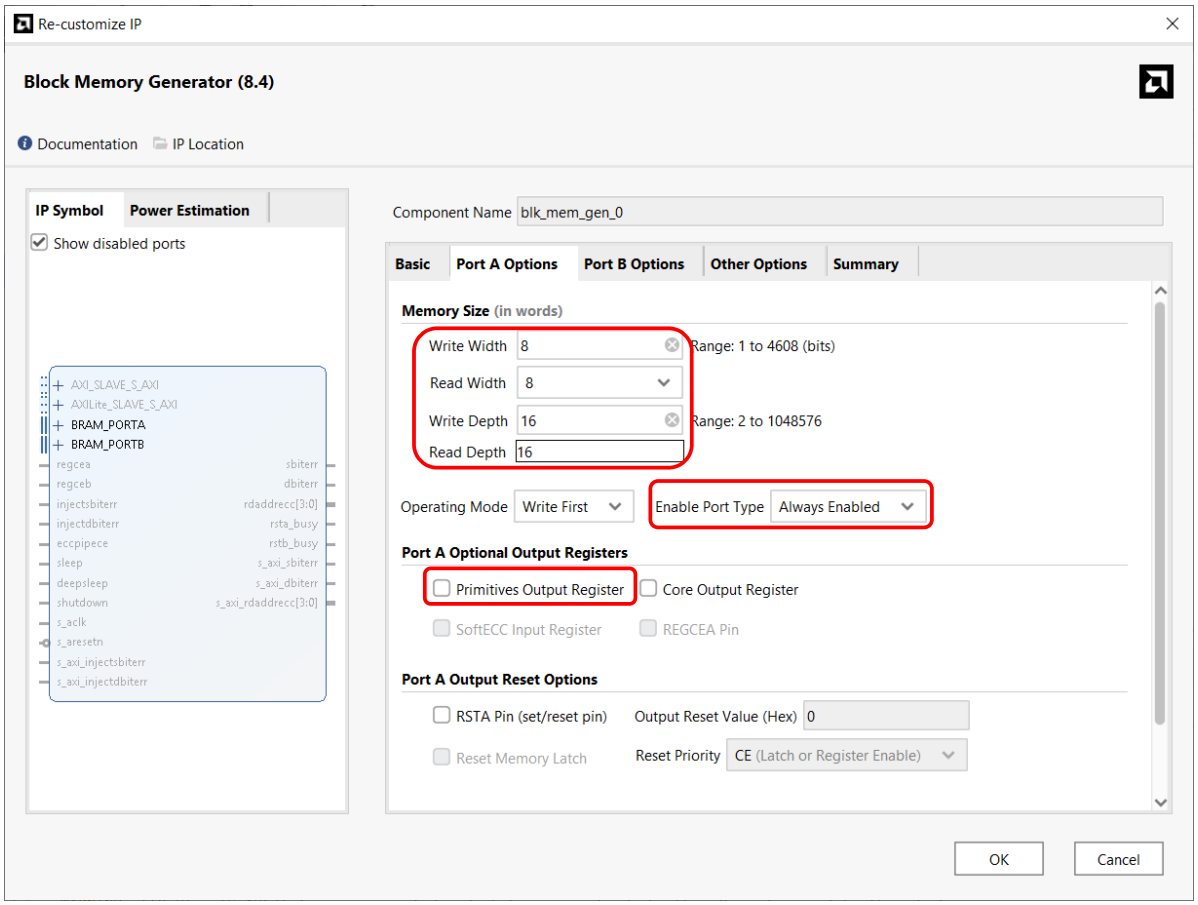
15. Zaznaczamy wygenerowany blok lewym przyciskiem myszy (należy zwrócić uwagę czy zaznaczył się cały blok czy tylko jakaś jego część), a następnie prawym przyciskiem wywołujemy jego menu kontekstowe, z którego wybieramy ***Customize Block...***



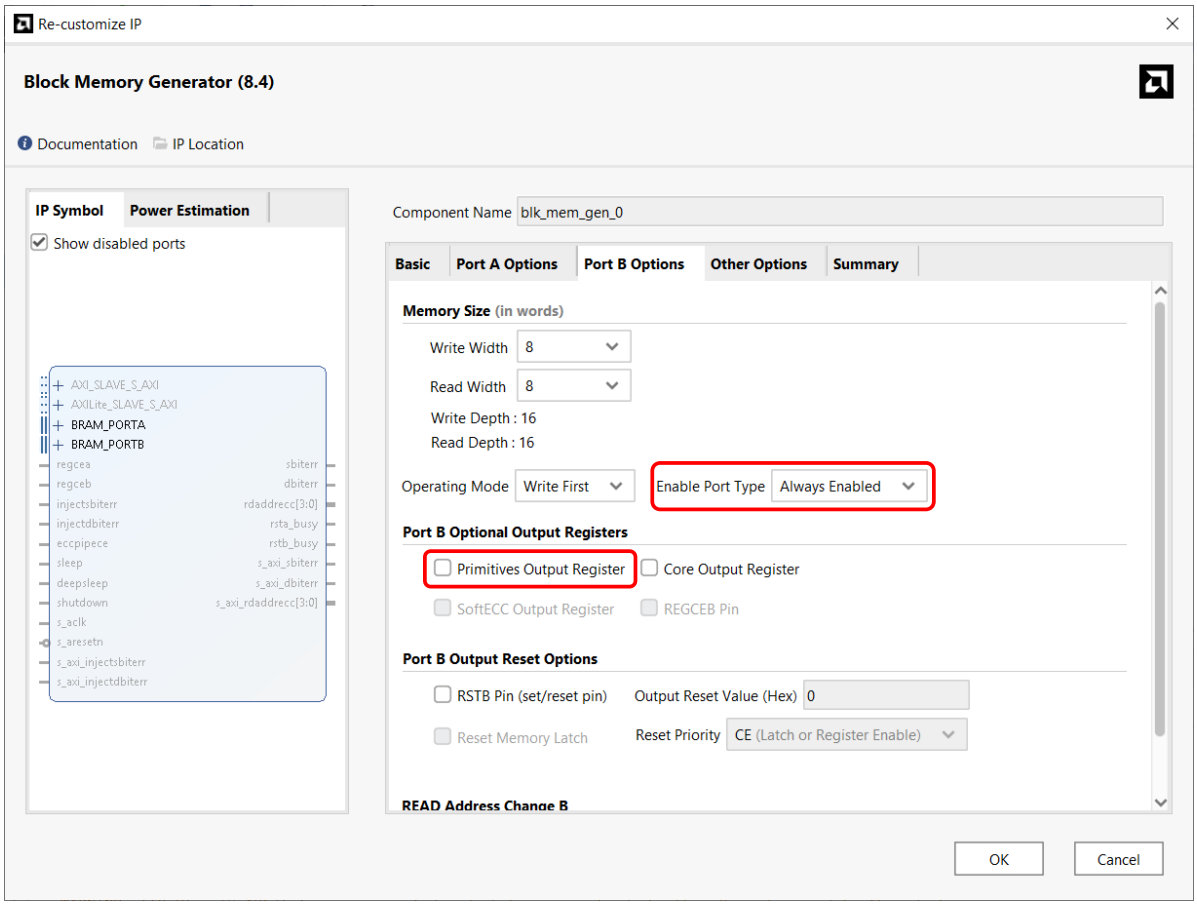
16. W zakładce ***Basic*** wybieramy tryb ***Stand Alone*** oraz typ pamięci: ***True Dual Port RAM***. Widzimy, że symbol bloku IP po lewej stronie okna zmienia się adekwatnie do wprowadzanych ustawień. Przechodzimy do zakładki ***Port A Options***.



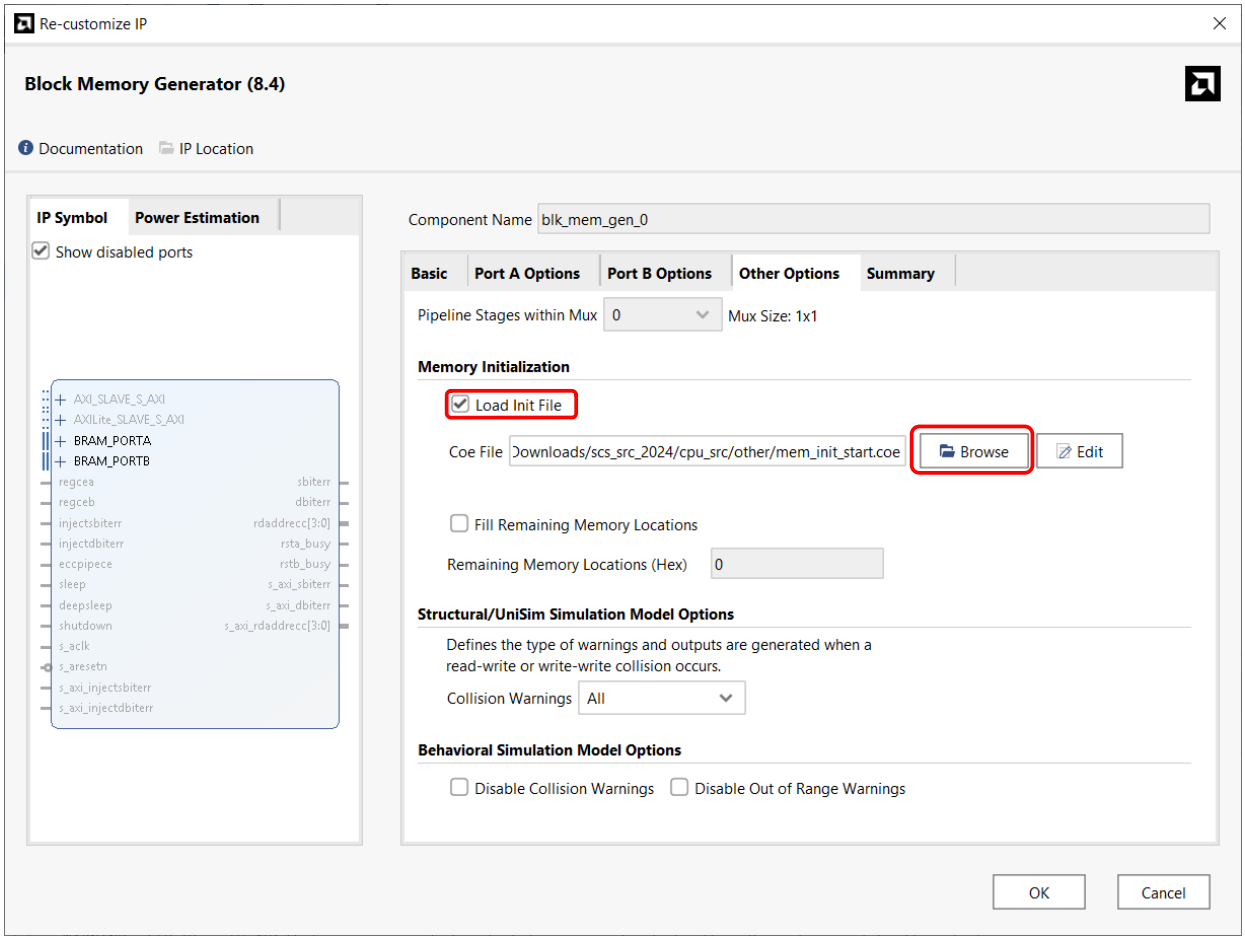
17. W zakładce *Port A Options* zmieniamy opisane dalej parametry.
    a) *Write Width* (szerokość zapisywanego słowa) ustawiamy na **8 bitów**,
    b) *Read Width* powinien zmienić się automatycznie również na **8 bitów**.
    c) *Write Depth* (ilość słów pamięci) zmieniamy na **16**.
    d) *Read Depth* również zmieni się adekwatnie.
    e) *Operating mode* pozostawiamy na ***Write First***
    f) *Enable Port Type* ustawiamy na ***Always Enabled***
    g) **Odznaczamy** opcję ***Primitives Output Register***
    h) Przechodzimy do zakładki ***Port B Options***



18. W zakładce Port B Options pozostało:
    a) Ustawić *Enable Port Type* na ***Always Enabled***
    b) **Odznaczyć** opcję ***Primitives Output Register***
    c) Przejść do zakładki ***Other Options***

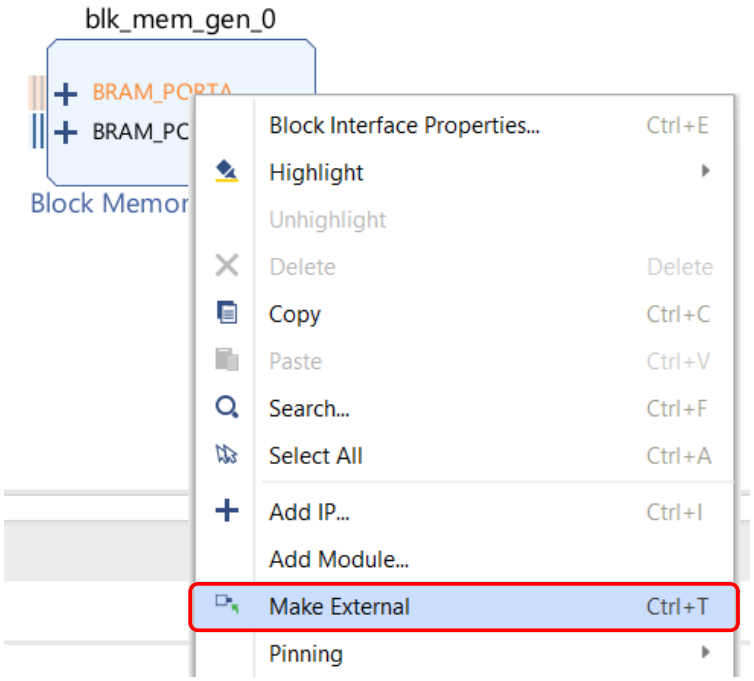


19. W zakładce *Other Options* wybieramy opcję ***Load Init File*** i po wciśnięciu przycisku ***Browse*** wskazujemy plik z testową zawartością pamięci naszego systemu mikroprocesorowego, w naszym przypadku ***mem_init_start.coe*** z podkatalogu ***other***.

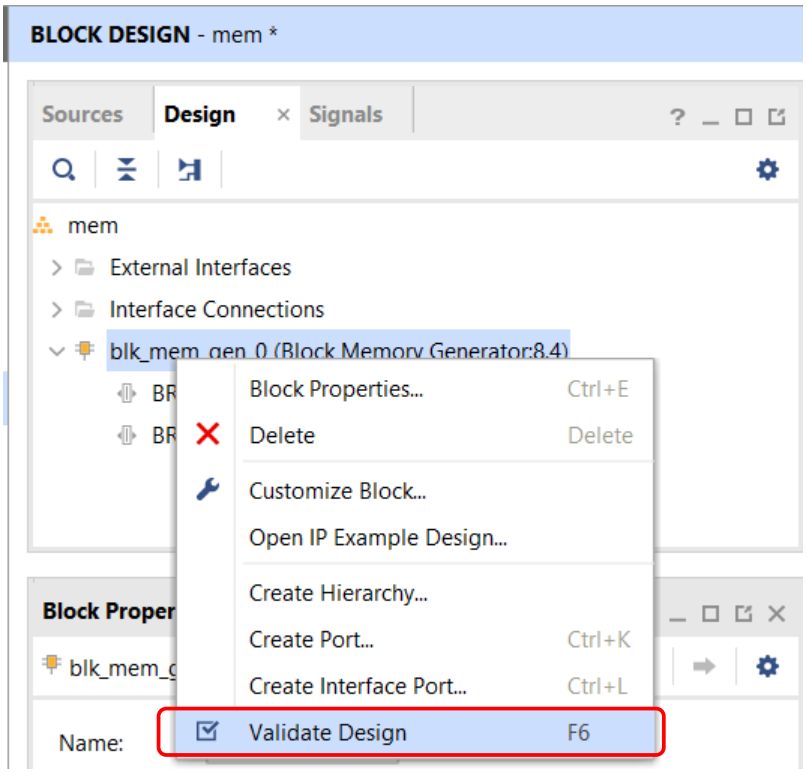


20. Po wybraniu pliku .coe zatwierdzamy ustawienia bloku IP klikając ***OK*** w oknie *Re-customize IP*.

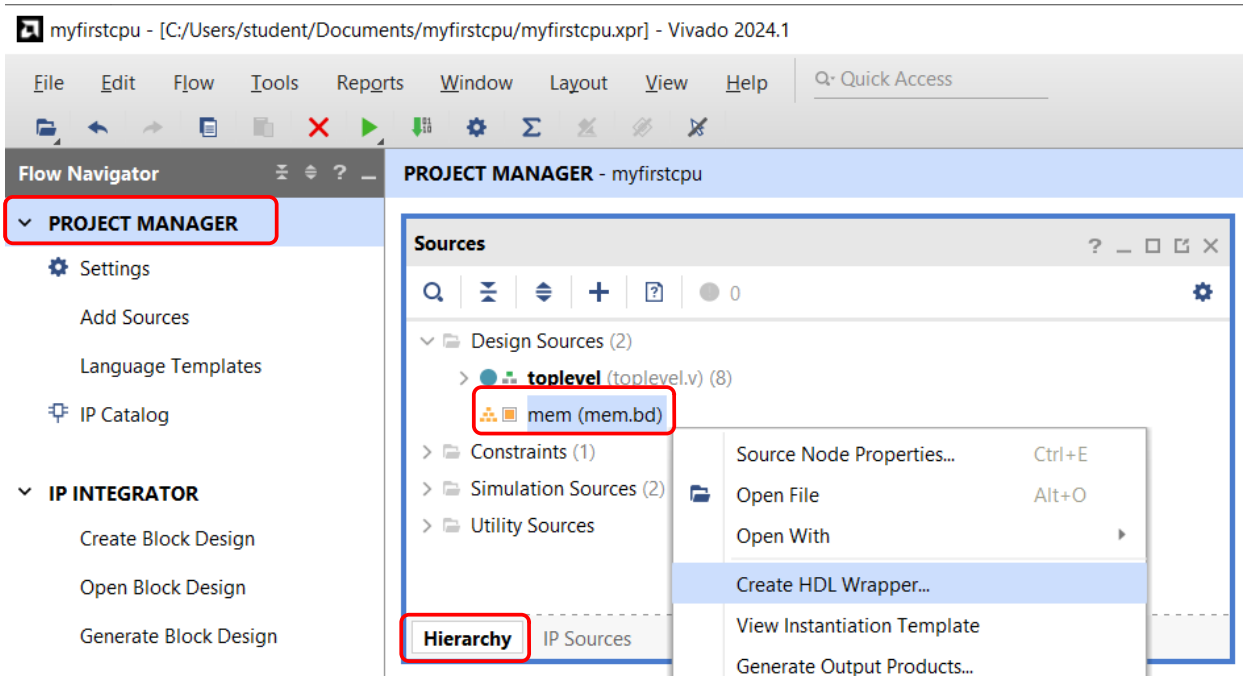
21. Powróciliśmy do edytora schematu blokowego. Klikamy w nim prawym klawiszem na jeden z portów ***(BRAM_PORTA)*** bloku IP stanowiącego pamięć operacyjną naszego systemu mikroprocesorowego i z menu kontekstowego wybieramy ***Make External***. **To samo robimy** dla drugiego portu pamięci ***(BRAM_PORTB)***. Zapisujemy schemat blokowy skrótem klawiszowym Ctrl+S.



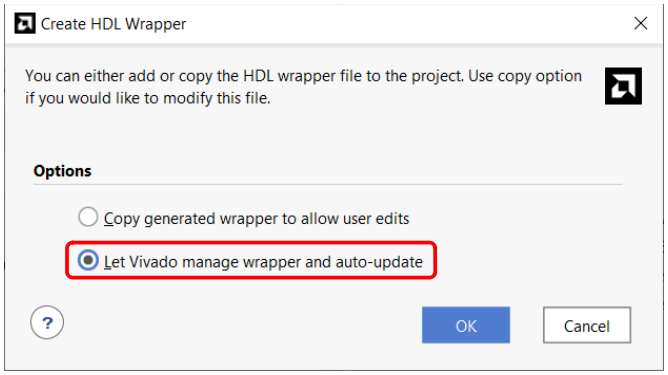
22. W obszarze *Design* edytora *Block Design* klikamy prawym przyciskiem myszy na ***blk_mem_gen_0*** i, z menu kontekstowego, wybieramy ***Validate Design***. Jeśli sprawdzenie nie zakończyło się błędami przechodzimy dalej.



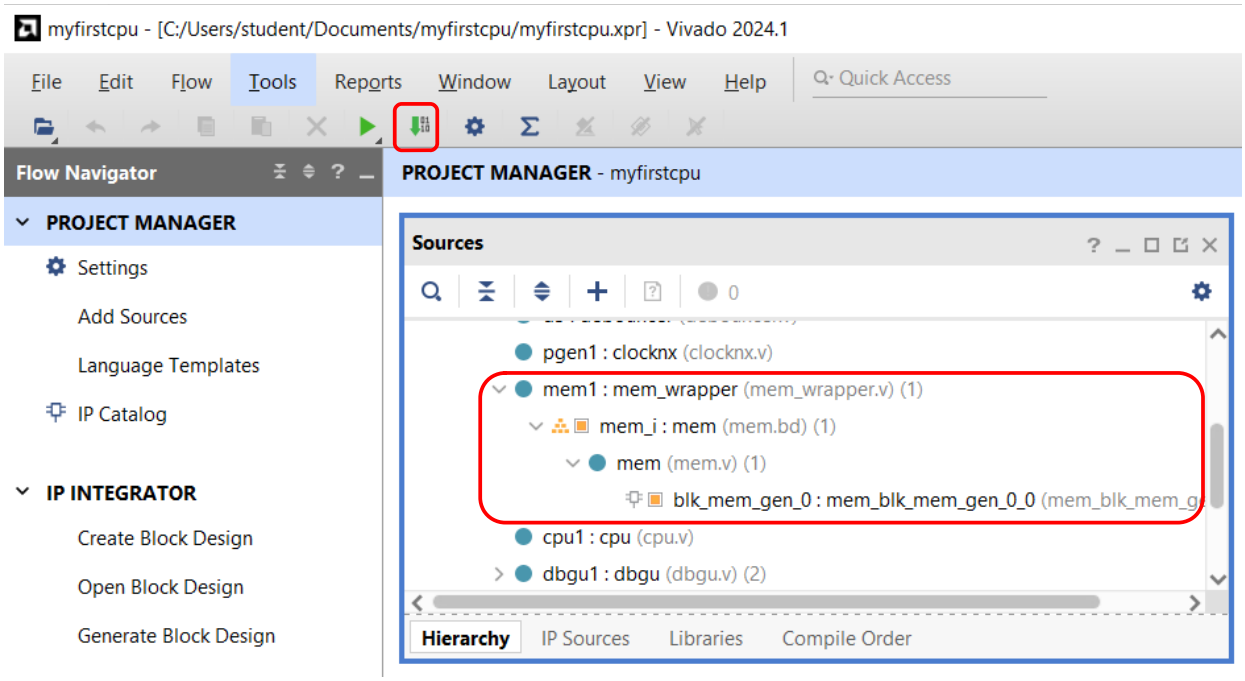
23. Przechodzimy do *Project Manager* i w polu *Sources* klikamy prawym klawiszem na element *mem*. Z menu kontekstowego wybieramy ***Create HDL Wrapper...***



24. W okienku dialogowym *Create HDL Wrapper* zalecane jest wybranie opcji *Let Vivado manage wrapper and auto-update*.

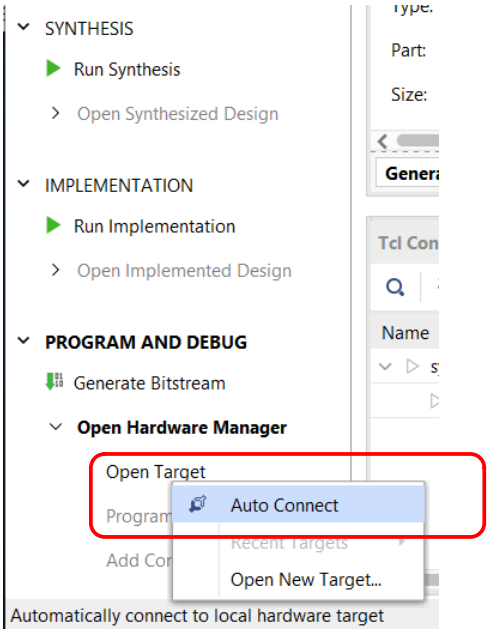


25. W obszarze ***Sources*** po rozwinięciu hierarchii *mem1* powinniśmy zobaczyć podobny widok jak na poniższym rysunku. Oznacza to, że właśnie wygenerowany moduł pamięci został prwidłowo rozpoznany przez środowisko Vivado. Jeśli nie widzimy błędów – klikamy ikonkę ***Generate Bitstream***.

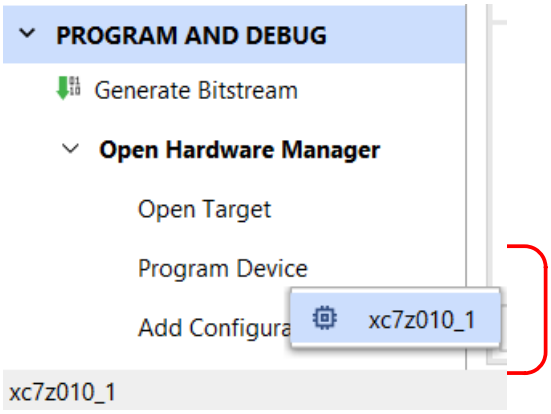


26. Jeśli środowisko Vivado zapyta, czy zapisać projekt przed kompilacją, oczywiście potwierdzamy to przyciskiem ***Save***. W oknie dialogowym z pytaniem, czy najpierw Vivado ma wykonać syntezę i implementację projektu, odpowiadamy ***Yes***.
27. Po zapisaniu projektu może się pojawić okno dialogowe z pytaniem o przydział zasobów komputera na potrzeby kompilacji. Należy zatwierdzić proponowane przez Vivado ustawienia.
28. Wszystkie etapy kompilacji mogą długo trwać (kilka lub nawet kilkanaście minut), co jest normalne i w dużej mierze uzależnione od zasobów i mocy obliczeniowej komputera, na którym pracujemy. Można teraz upewnić się, czy płytka Zybo na pewno jest podłączona do złącza USB komputera, na którym pracujemy oraz, czy kabel dodatkowego łącza szeregowego jest poprawnie podłączony do płytki. **W razie jakichkolwiek wątpliwości należy skonsultować konfigurację sprzętową z osobą prowadzącą zajęcia**.
29. Po zakończeniu kompilacji i generowania plików wynikowych może pojawić się okno dialogowe *Bitstream Generation Completed*. Na tym etapie nie korzystamy z żadnych oferowanych w nim opcji, więc możemy wcisnąć przycisk *Cancel* lub zamknąć je klawiszem ESC.

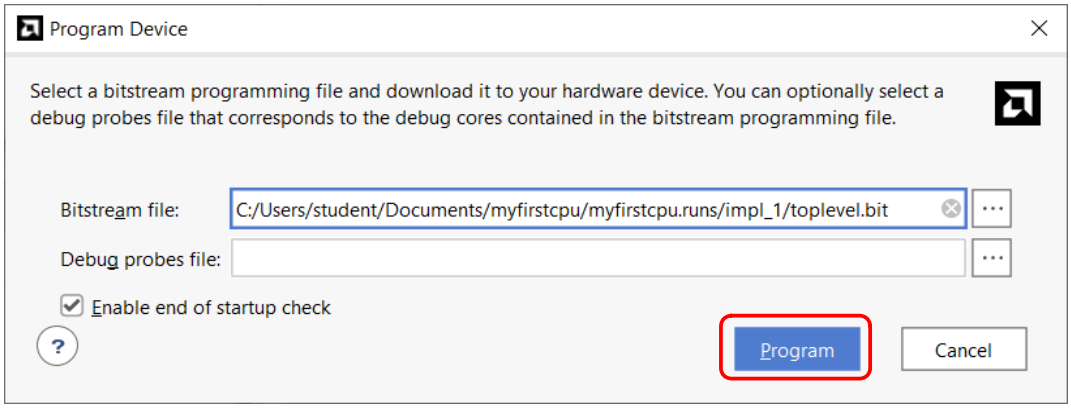
30. Na samym dole lewej części okna środowiska Vivado rozwijamy listę *Open Hardware Manager*, a następnie klikamy ***Open Target*** i ***Auto Connect***.



31. Z tego samego obszaru wybieramy Program Device i xc7z010_1 (jedyna dostępna opcja).



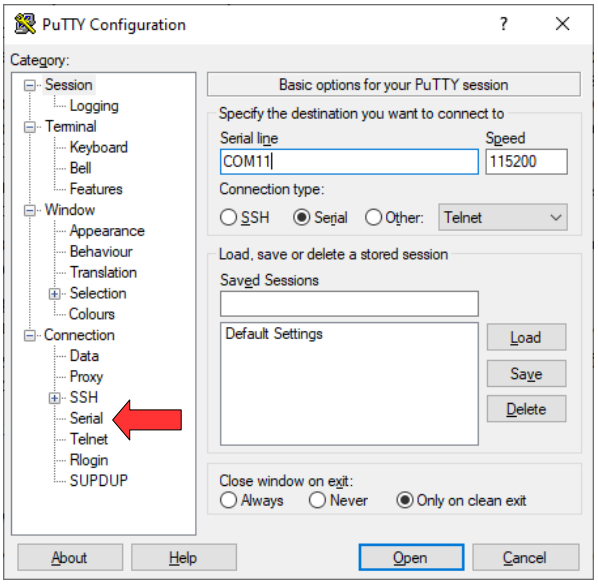
32. W oknie dialogowym Program Device pozostajemy przy domyślnie zasugerowanym pliku .bit i wciskamy przycisk **Program**.

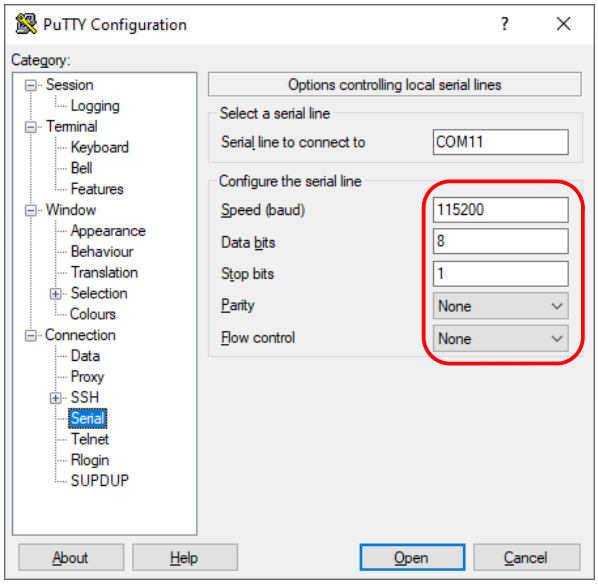


3.3 . Nawiązanie komunikacji przez port szeregowy

1. Upewniamy się, że przewód USB z modułu PMOD USB-UART znajduje się w złączu USB komputera na stole roboczym.
2. W przypadku pracy z systemem Windows sprawdzamy przy użyciu Menedżera Urządzeń jakie porty COM inne niż COM1 i COM3 funkcjonują w systemie. Zwracamy uwagę na porty opisane jako ***USB Serial*** lub podobnie.
3. Otwieramy program terminalowy, np. ***putty*** lub ***teraterm*** i łączymy się z wykrytym w poprzednim punkcie wirtualnym portem COMxx generowanym przez sterownik modułu PMOD. Jeśli portów jest więcej, należy sprawdzić każdy z nich lub w inny sposób ustalić który port odpowiada za współprace z modułem PMOD USB-UART (np. odłączając na chwilę od komputera połączony z nim przewód USB, monitorując jednocześnie, który z portów COM znika z systemu).

   Poprawne parametry transmisji to: prędkość **115200 baud, 8 bitów danych, 1 bit stopu, brak bitu parzystości**. Konfigurację PuTTY dla przykładowego portu COM11 pokazano na rysunkach:





4. Po wyborze parametrów połączenia w putty wciskamy przycisk ***Open***. Połączenie powinno teraz zostać nawiązane.
5. Z zaprogramowanym układem FPGA, wciskamy na płytce Zybo przycisk BTN[2] przy przełącznikach SW[3:0] ustawionych np. na 0101 (od lewej: „dół-góra-dół-góra”).
6. Jeśli wszystko wykonane zostało poprawnie, na terminalu powinniśmy po każdym wciśnięciu przycisku BTN[2] ujrzeć jedną lub więcej linijkę tekstu np.

   c=1 r0=1 r1=0 pc=3 A=3 R=81 W=00 M: 80 90 81 92 A0 00 00 00 00 00 00 00 00 00 00 00

3.4 . Obserwacja działania programu testowego

Po zaprogramowaniu układu FPGA przechodzimy do obserwacji prostego programu domyślnego.

1. Włączamy „ręczne” podawanie sygnału zegarowego ustawiając przełącznik SW[1] w pozycji „w dół”,
2. Zerujemy system przyciskiem BTN[0].
3. Obserwujemy wykonanie poszczególnych instrukcji wciskając przycisk BTN[2]. Przy ustawieniach przełączników SW[3:0] na 0100 (od lewej „dół-góra-dół-dół”), mikroprocesor będzie wykonywał po jednej pełnej instrukcji, ponieważ jedno wciśnięcie BTN[2] odpowiada podaniu do mikroprocesora 4 okresów sygnału zegarowego.
4. Obserwujemy zawartość rejestrów: PC, R0, R1 oraz danych odczytywanych z pamięci. Porównujemy dane odczytane z pamięci z zawartością pliku ***mem_init_start.coe***. Zawartość poszczególnych fragmentów każdej linijki obserwowanej na terminalu jest następująca:
   - c: wewnętrzny cykl mikroprocesora. Jeden taki cykl odpowiada jednemu cyklowi (okresowi) sygnału zegarowego podanego do mikroprocesora. Cykl instrukcji składa się z 4 cykli zegarowych,
   - r0, r1, pc: zawartość głównych rejestrów mikroprocesora w formacie szesnastkowym,
   - A: dane na magistrali adresowej pamięci,
   - W: dane na magistrali zapisu z procesora do pamięci,
   - R: dane na magistrali odczytu z pamięci do procesora,
   - M: zrzut wszystkich komórek pamięci (jest ich 16).
5. Możemy bez trudu zaobserwować, co dzieje się w poszczególnych stanach mikroprocesora w czasie wykonywania pojedynczej instrukcji i odnieść swoje obserwacje do treści pliku ***cpu.v***.

3.5 . Pisanie własnego programu

Korzystając z nabytych do tej pory umiejętności oraz z dokumentacji mikroprocesora (***cpu_dok.pdf***) należy samodzielnie napisać (w kodzie maszynowym) program, który będzie wykonywał wymienione poniżej czynności.

Dla wygody możemy tymczasowo utworzyć kopię pliku *mem_init_start.coe* i modyfikować ją przy pomocy dowolnego edytora tekstowego. Utworzonej kopii jednak nie będziemy używać bezpośrednio, tylko przepiszemy jej treść używając na dalszych etapach edytora pamięci w GUI środowiska Vivado. Oczywiście równie dobrze można użyć dowolnego innego sposobu tworzenia

kodu „na brudno”, by później wpisać go w edytorze pamięci Vivado.

Zaczynamy od wpisania liczby ***0x05*** do komórki pamięci o adresie **13** – będzie to jedna z danych wejściowych do obliczeń w programie. Program ma wykonać następujące czynności:

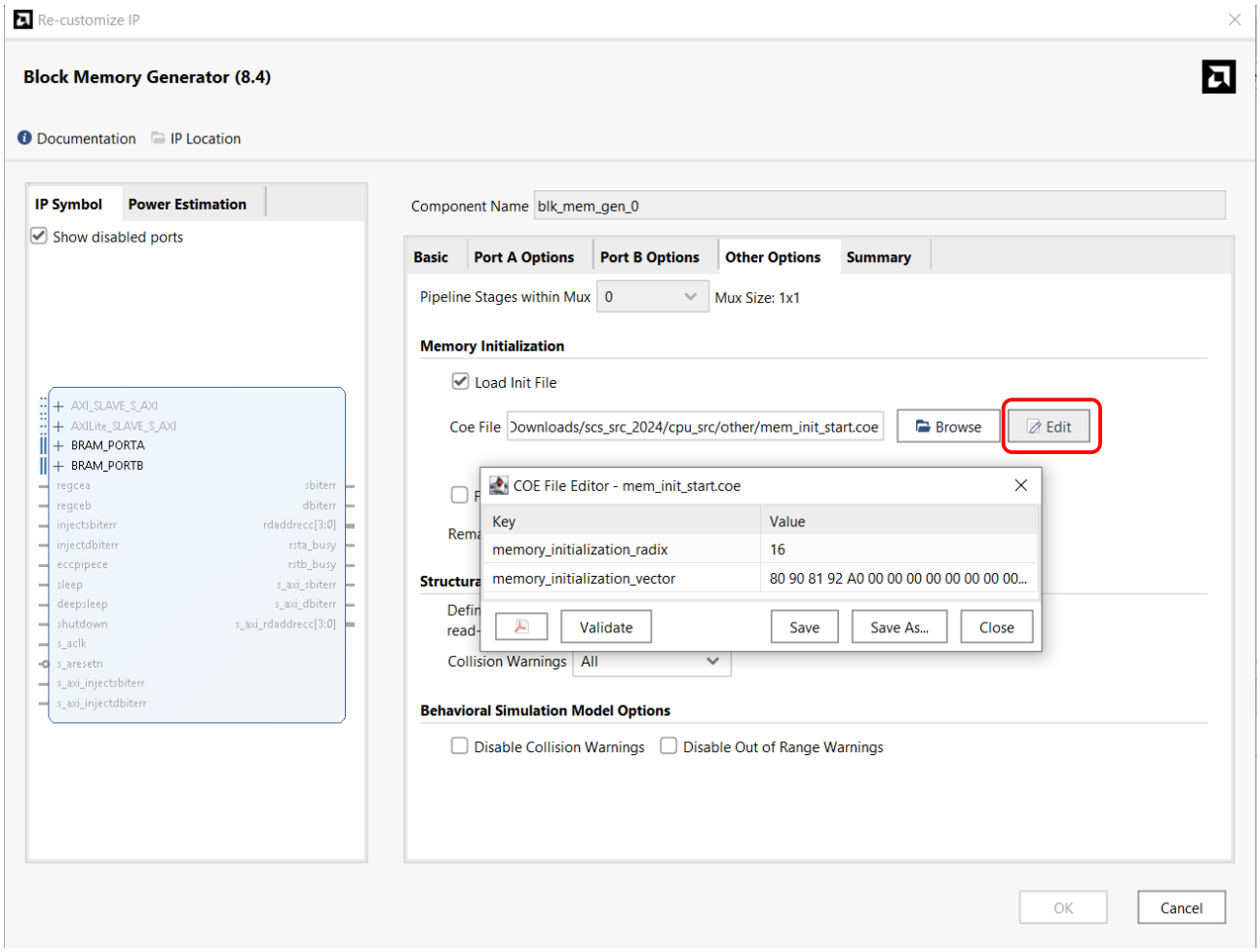
1. Wyczyścić zawartość rejestrów R0 i R1 wpisując do nich wartość 0,
2. Wyczyścić zawartość komórki pamięci o adresie 14 wpisując do niej wartość 0,
3. Do rejestru R0 wpisać liczbę 0x6
4. Dodać do rejestru R0 wartość przechowywaną w pamięci RAM pod adresem 13,
5. Umieścić zawartość rejestru R0 w pamięci RAM w komórce o adresie 14,
6. Zapętlić wykonywanie programu.

Program należy uruchomić i zaprezentować jego działanie osobie prowadzącej zajęcia.

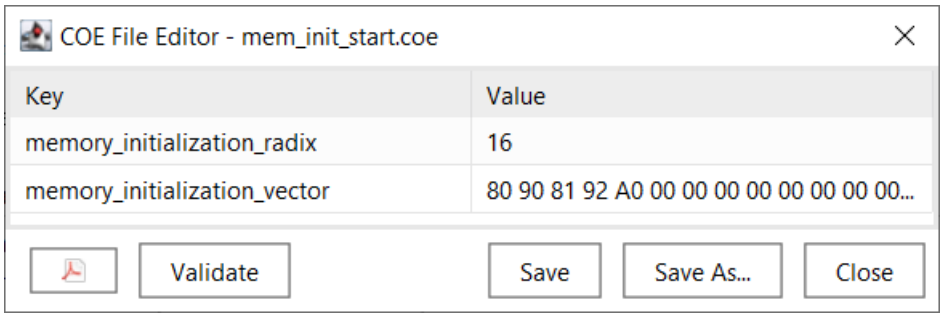
Uwaga: aby nowy program znalazł się w systemie mikroprocesorowym w układzie FPGA należy wykonać poniższe kroki. W razie problemów z umieszczeniem nowego kodu w pamięci systemu, należy poprosić o pomoc osobę prowadzącą.

W oknie Flow Nawigator, w sekcji IP INTEGRATOR, poleceniem ***Open Block Design*** otwieramy ponownie projekt utworzonego wcześniej podsystemu pamięci.

Zaznaczamy blok pamięci i z menu kontekstowego wybieramy **Customize block…** . W otwartym właśnie oknie własności bloku pamięci przechodzimy do zakładki **Other Options** i tam przyciskiem **Edit** uruchamiamy edytor zawartości pliku *.coe.



W oknie ***COE File Editor*** klikamy na znajdującą się w zaznaczonym obszarze wartości, którą chcemy zmodyfikować i zastępujemy ją nową.



Po zakończeniu edycji wciskamy kolejno przyciski: ***Save***, ***Validate***, ***Close***. A następnie przyciskiem OK zamykamy edytor właściwości bloku pamięci.

Po wykonaniu tych czynności, zapisujemy cały projekt i ponownie klikamy ***Generate Bitstream***.

Po zakończeniu kompilacji ponownie wgrywamy konfigurację do układ FPGA i przechodzimy do obserwacji działania nowego programu

3.6 . Utworzenie nowej instrukcji mikroprocesora

Rozszerzyć możliwości instrukcji dodawania w taki sposób, by można było wybrać, do którego rejestru (R0 czy R1) wpisywany będzie wynik. Wybór rejestru powinien odbywać się przez zmianę wartości bitu 4 kodu operacji.

Następnie dodać nową instrukcję wykonującą wybrane działanie. Może to być np. mnożenie, odejmowanie lub jeszcze inna operacja **uzgodniona z osobą prowadzącą zajęcia**.

Prezentację działania nowej instrukcji należy wykonać na płytce testowej modyfikując odpowiednio program napisany w punkcie 3.6 i prezentując jego działanie.

4 . Co należy umieć i/lub co należy uwzględnić w sprawozdaniu

Wymienione w niniejszej sekcji zagadnienia należy opracować samodzielnie, a następnie dla każdego punktu odnieść się zwięźle w sprawozdaniu i/lub utrwalić zdobytą wiedzę i umiejętności zależnie od ogłoszonej przez osobę prowadzącą formy weryfikacji efektów uczenia dla tego zadania.

1. Rozróżnienie cykli zegarowych i cykli instrukcji w mikroprocesorze dydaktycznym z ćwiczenia *cpu* oraz w innych mikroprocesorach.
2. Klasyfikacja mikroprocesora, na którym wykonywane jest ćwiczenie oraz umiejętność uzasadnienia takiej klasyfikacji pod kątem:
   a) organizacji ścieżek danych: Harvard / von Neumann
   b) zestawu i kodowania instrukcji: RISC / CISC
   c) wspieranej szerokości bitowej jego najważniejszych elementów
3. Proszę omówić w kontekście wsparcia dla konkretnych klas instrukcji podstawowe możliwości mikroprocesora dydaktycznego w bazowej konfiguracji:
   a) klasa: instrukcje skoków – które rodzaje instrukcji skoków może wykonywać?
   b) klasa: instrukcje przesłań – które rodzaje przesłań wspiera, tj. pomiędzy którymi elementami? (np. czy rejestr-rejestr, czy pamięć-pamięć, czy rejestr-pamięć itd.)
   c) klasa: instrukcje przetwarzania danych – które rodzaje obliczeń może wykonywać? (np. które operacje arytmetyczne wspiera?)
4. Proszę wymienić i zwięźle opisać zastosowanie rejestrów wewnętrznych, w które jest wyposażony mikroprocesor, oraz rozmiaru (szerokości bitowej) tych rejestrów.
5. Proszę zwięźle omówić parametry wydajnościowe mikroprocesora dydaktycznego:
   a) czy ma wsparcie dla przetwarzania potokowego?
   b) jaka jest liczba cykli zegarowych przypadających na wykonanie jednej instrukcji i czy liczba ta jest stała dla każdej instrukcji, czy zmienia się zależnie od wykonywanej instrukcji?
   c) jak nazywają się poszczególne etapy, z których składa się wykonanie jednej instrukcji?